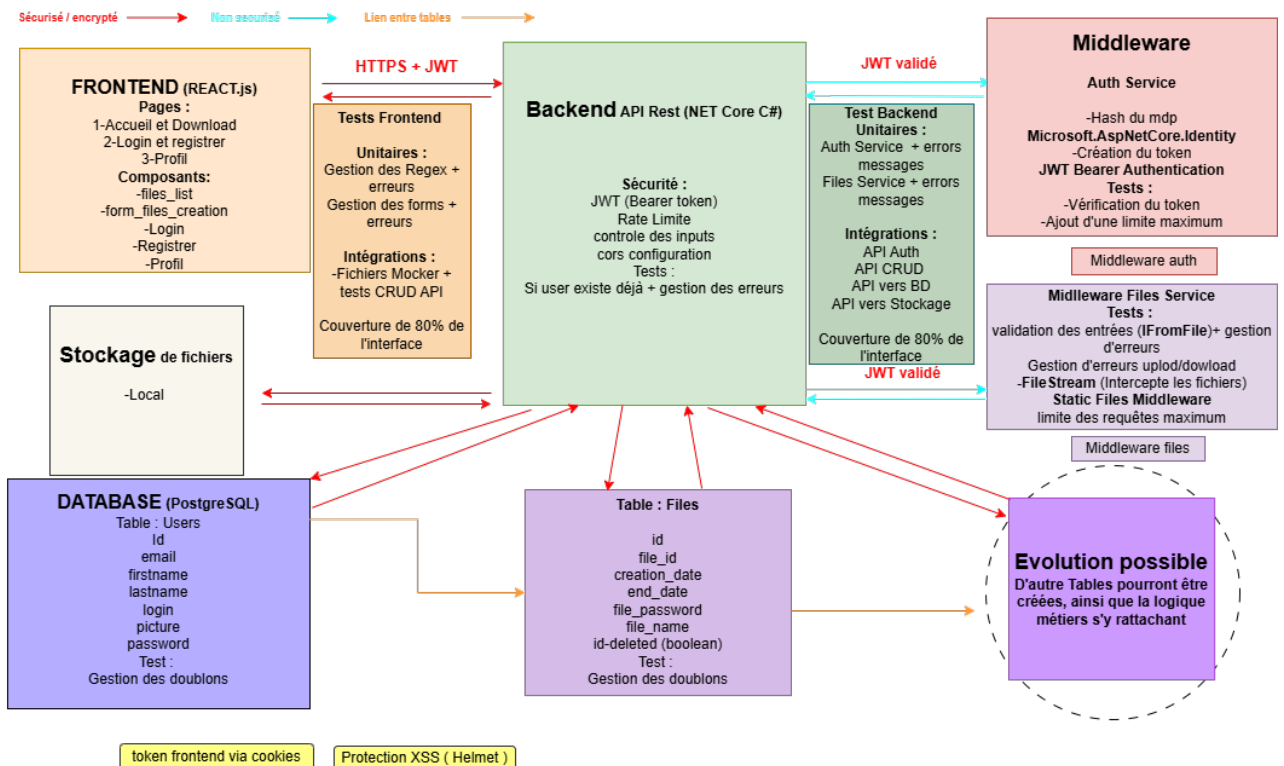


Modèle de documentation technique

<u>Architecture de l'application (diagramme simple).....</u>	<u>2</u>
<u>Choix technologiques justifiés (1-2 pages).....</u>	<u>2</u>
<u>Modèle de données.....</u>	<u>4</u>
<u>Documentation d'API.....</u>	<u>5</u>
<u>Sécurité et gestion des accès.....</u>	<u>6</u>
<u>Qualité, tests et maintenance.....</u>	<u>6</u>
<u>Processus d'installation et d'exécution.....</u>	<u>6</u>
<u>Utilisation de l'IA dans le développement.....</u>	<u>7</u>

Architecture de l'application (diagramme simple)

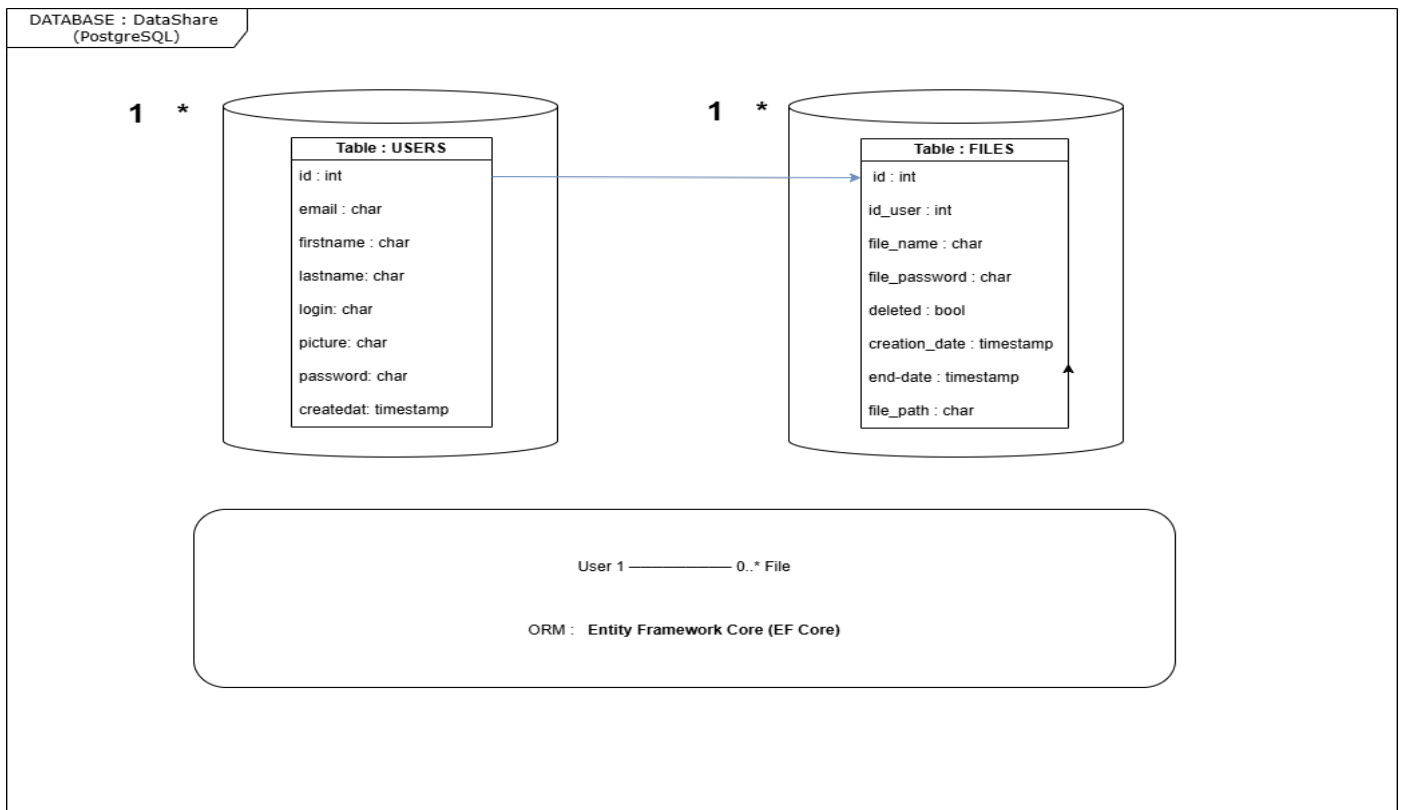


Choix technologiques justifiés (1-2 pages)

Backend	Framework .net langage C#	Node.js	Interopérabilité du framework et multitudes de possibilités d'intégrations dans un environnement d'entreprise. Le framework Node.js par exemple est moins structuré.
Frontend	React.js	Angular	Typé avec typescript permet une écriture du code structuré. Angular est aussi une bonne alternative car nativement typé.
Base de donnée	Postgre SQL	MySQL/MongoDB	PostgreSQL est libre de droit et dispose d'une communauté open source solide.
Authentification	ASP.NET Identity	Active Directory Azure Ad	La bibliothèque .net identity dispose de modèles robustes pour l'identification, elle est fournie par Microsoft et peut par la suite être intégrée à Active Directory ou Azure AD.

Migration de table	Entity Framework Core (code first) (ORM)	...	Gère la logique d'écriture en langage SQL depuis l'application vers la base de données grâce a un modèle écrit en C#, qui peut être intégrer sur AAD.
Stockage des fichiers	Dossier locaux	Amazon AWS S3 (Buckets)	Les liens des fichiers de la base de données pointent sur un chemin local. Simple a mettre en place pour un MVP, l'application pourra ensuite être connectée si besoin a AWS S3 et écrire dans des buckets + une gestion des droits IAM
Hébergement	Démarrage de l'application en localhost	Hébergement sur un serveur web.	Ne nécessite pas de coût d'hébergement pour un d'environnement de développement et de tests.
CI/CD	Tests Xunit et Jest	Nunit et Jasmine	Test backend réalisés avec Xunit, dans un environnement de test dédié. Tests frontend réalisés avec la librairie Jest incluant la simulation d'un dom et sans le dom.
Outils de développement	Visual Studio et Visual Studio Code	JetBrains Rider et Cursor (IA)	Visual Studio pour le backend afin d'obtenir un environnement dédié a .net et Visual studio code pour le développement frontend.
Outils de développement	Github et Git bash	Gitlab	Versioning de l'application

Modèle de données



Documentation d'API

L'API DataShare est une API REST développée en **ASP.NET Core**, permettant la gestion des utilisateurs et le partage sécurisé de fichiers.

Elle est consommée par une application frontend développée en **React.js Typescript**.

L'API respecte les principes REST : -Utilisation des verbes HTTP standards (GET, POST, PUT, DELETE)

-Echanges de données au format JSON & -Authentification sécurisée par JWT ici stocké en cookie HttpOnly

Sécurité et gestion des accès

-Json Web Token

-Hash du mots de passe

-Protocole SSL, certificat approuvé par ASP.NET core pour le chiffrement des connexions

-Token écrit dans les cookies du navigateur.

-Validations des entrées

-Taille maximal d'upload de 1Gb

Qualité, tests et maintenance

Résumé du plan de maintenance

Le plan de maintenance du projet DataShare définit les procédures essentielles pour garantir la stabilité, la sécurité et la performance de l'application frontend (React/TypeScript) et backend (ASP.NET Core). Il couvre la mise à jour régulière des dépendances, le nettoyage des environnements, l'exécution systématique des tests, la gestion de la sécurité (mots de passe, sauvegardes PostgreSQL, HTTPS) ainsi que le suivi des performances et des logs. Des bonnes pratiques de versioning, de documentation et de suivi des évolutions sont appliquées afin d'assurer une maintenance continue et fiable du projet.

- *TESTING.md (plan de tests, critères d'acceptation, instructions d'exécution)*
- *SECURITY.md (résultats du scan de sécurité + analyse des décisions)*
- *PERF.md (test de performance d'un endpoint avec résultats et interprétation, budget de performance côté front, captures de logs ou métriques de performance de navigateur et/ou de serveur)*
- *MAINTENANCE.md (procédures de mise à jour, fréquence, risques)*

Processus d'installation et d'exécution

Prérequis

Backend

.NET SDK 8.0 ou supérieur

PostgreSQL 12+ (base de données)

Frontend

Node.js 16.x ou supérieur

npm 8.x ou supérieur

Installation

Backend

bash

Naviguer vers le dossier backend

cd DataShareBackend

Restaurer les dépendances

dotnet restore

Appliquer les migrations de base de données

dotnet ef database update

Mode développement dotnet run

Mode production dotnet publish -c Release

dotnet DataShareBackend.dll

Frontend

bash

Naviguer vers le dossier frontend

cd frontend

Installer les dépendances

npm install

Mode développement (http://localhost:3000)

npm start

Build de production npm run build

Lancer les tests npm test

Variables d'environnement à configurer

```
{ "ConnectionStrings": { "DefaultConnection":
```

```
"Host=localhost;Database=datashare;Username=[USER];Password=[PASSWORD]" }, "Jwt": { "Key":
```

```
"[CLÉ_SECRÈTE_JWT]", "Issuer": "[ISSUER]", "Audience": "[AUDIENCE]", "ExpirationInMinutes": 60 }, "FileStorage":
```

```
{ "UploadPath": "[CHEMIN_STOCKAGE_FICHIERS]", "MaxFileSize": 10485760 } }
```

Utilisation de l'IA dans le développement

L'ia à été utiliser en guise de questionnements sur la mise en pratique du code sur les bugs ou pour avoir plusieurs exemples de bonne pratiques, elle n'a pas été a l'origine des prises de décisions concernant la conception de l'application.

Le gain de temps est effectifs lorsque on lui confit une tâche répétitive tel que l'injection du CSS mais si on le lui indique pas dès le départ de ne pas créer de répétition dans le code elle ne prendra pas l'initiative de le faire et répétera des morceaux de code qui aurait pu être simplifiés et optimisés.